

桃江经典王麻将 (TJMJ)

全栈实时 Web 平台技术手册

中文版 v3.0

JaiJaiC

湖南桃江

<https://github.com/JaijaiC/TJMJ>

2026 年 6 月 25 日

摘要

本文系统阐述了一款面向湖南桃江地区的全栈网页麻将游戏平台——TJMJ（桃江经典王麻将）的技术实现。平台前端采用 Vue 3 Composition API 单页架构、Vite 8 构建工具链，后端基于 Node.js WebSocket 服务器，语音通信采用 WebRTC P2P 网状拓扑辅以 Twilio TURN 中继。系统完整实现了桃江地方麻将规则体系：打筛扳王（癞子）机制、扎鸟翻倍计分、门子累加番型、组合胡法等。核心算法包括 $O(3^k)$ 复杂度的递归回溯胡牌判定引擎（实测亚毫秒级）、启发式 NPC 出牌策略、以及支持自定义拖拽的手牌管理系统。本文详述了系统架构设计、通信协议栈、游戏规则数学模型、核心算法实现、多平台兼容性方案及性能评测结果。

目录

1	引言	2
2	系统架构	2
2.1	技术栈	2
2.2	核心文件结构	2
2.3	前后端通信协议	2
3	游戏规则与数学模型	2
3.1	牌组编码	2
3.2	打筛扳王	2
3.3	发牌与游戏流程	2
3.4	操作优先级	2
3.5	胡法分类	2
3.6	计分模型	2
4	核心算法	3
4.1	胡牌判定引擎	3
4.2	听牌检测	3
4.3	NPC 出牌策略	3
4.4	手牌拖拽与同步	3
5	多人在线基础设施	3
5.1	容错机制	3
5.2	网络延迟实测	3
6	跨平台兼容性	3
6.1	iOS 适配详情	3
6.2	Android 适配详情	4
7	性能评测	4
8	讨论与展望	4
8.1	当前局限	4
8.2	未来方向	4
2	参考资源	4

1 引言

中国麻将起源于清代，经过数百年的地域演化形成了众多地方变种。湖南桃江地区的经典王麻将是其中独具特色的一支：使用精简的 108 张牌组（万/筒/条，不含风牌与花牌）、通过打筛扳王动态确定万能牌（癞子）、采用扎鸟机制在胡牌后进行翻倍计分、允许多种胡法累加番型。

尽管市场上存在诸多麻将类应用，但无一能完整满足桃江玩家的需求。闲逸等商业 App 要求强制注册登录、界面繁琐、规则与本地玩法存在差异、缺少拖拽手牌和实时语音等现代化交互功能。更为重要的是，现有平台均未提供可供二次开发的开放源代码基础。

TJMJ 应运而生——一款无需安装、点击链接即可进入、规则与桃江本地高度一致的网页麻将平台。项目采用全开源架构，前端部署于 GitHub Pages，后端通过 ngrok 公网穿透。总计约 4200 行核心 JavaScript 代码，涵盖单人 AI、四人联机、观战三大模式。

2 系统架构

2.1 技术栈

2.2 核心文件结构

2.3 前后端通信协议

客户端与服务端通过单一 WebSocket 长连接进行 JSON 消息通信。协议共定义 18 条客户端到服务端消息和 12 条服务端到客户端消息。

3 游戏规则与数学模型

3.1 牌组编码

麻将牌组 $\mathcal{T}$ = $\{11, \dots, 19, 21, \dots, 29, 31, \dots, 39\}$ ，共 108 张。编码规则： $t = 10s + v$ ，其中 $s \in \{1, 2, 3\}$ 为花色（1=万, 2=条, 3=筒）， $v \in \{1, \dots, 9\}$ 为面值。每张牌有 4 个副本。

将牌（ $\mathcal{J}$ ）定义为面值属于 $\{2, 5, 8\}$ 的牌，共 $3 \times 3 \times 4 = 36$ 张。将牌是胡牌的必要组成部分。

3.2 打筛扳王

开局由庄家掷两枚骰子 d_1, d_2 ，点数之和 $S = d_1 + d_2$ 。从庄家 P_0 开始逆时针数到第 S 家（ P_0 对应 1,5,9），该玩家面前的牌墙为起手墙。在起手墙右手边起跳过 d_1 叠后开始抓牌。

扳王规则：从起手墙对应玩家的下家墙左手边数到第 d_2 叠，翻开最上面一张牌即为”地”（ T_d ）。”王”（癞子， T_w ）的计算公式为：

$$T_w = \begin{cases} \lfloor T_d/10 \rfloor \times 10 + 1, & \text{若 } T_d \bmod 10 = 9 \\ T_d + 1, & \text{否则} \end{cases} \quad (1)$$

3.3 发牌与游戏流程

初始发牌共 53 张：前 48 张每人 12 张，随后庄家补 1 张、闲家各补 1 张（各 13 张），庄家再补 1 张（共 14 张）。游戏开始后，回合按逆时针方向进行。每位玩家先摸一张牌，再打出一张牌。其他玩家可对该打出的牌进行拦截操作。

3.4 操作优先级

1. 胡一点炮或自摸，直接获胜（最高优先级）
2. 碰/杠一手中有相同牌时碰之或杠之
3. 吃一仅能吃上家打出的牌构成顺子

联机模式中若同时满足吃和碰，两个按钮同时亮起供玩家选择。抢杠规则：明杠时若该牌恰好是他人要胡的牌，杠家全赔。

3.5 胡法分类

胡牌基本结构为：一对将 + 四句话，每句话可为刻子（AAA）或顺子（ABC）。门子（额外胡法）采用累加番制。

3.6 计分模型

自摸得分公式为：

$$\Sigma = B \cdot \left(\sum m_k \right) \cdot Z \cdot G \quad (2)$$

其中 $B = 3$ 为底分， $\sum m_k$ 为所有门子番数之和， $Z \in \{1, 2\}$ 为扎鸟倍数， G 为杠倍数。

扎鸟机制：胡牌后翻开下一张牌，根据其个位数字 v 判定翻倍对象：

- $v \in \{1, 5, 9\}$: 三家全中, 全部翻倍
- $v \in \{2, 6\}$: 下家翻倍
- $v \in \{3, 7\}$: 对家翻倍
- $v \in \{4, 8\}$: 上家翻倍

4 核心算法

4.1 胡牌判定引擎

胡牌检测采用递归回溯算法, 核心流程如下:

步骤1: 分离王牌(nw)和普通牌(N), 统计地牌数量(nd)

步骤2: 特殊判定: 黑天胡(首轮14张无王无将无句)

顶级判定: 4王→天天胡, 3王→天胡, 3地→地胡

步骤3: 递归检查常规胡牌(刻子/顺子/将)

步骤4: 检查七小对

步骤5: 门子判定: 将将胡→清一色→碰碰胡→硬庄→平胡重新加入。

递归子程序每次选取最小牌值 $t_{\min}$, 尝试三种消除: 刻子、顺子、将(对子)。当普通牌不足以匹配时消耗王牌替代。递归深度 $k \leq 5$, 复杂度 $O(3^k)$, 实测平均耗时 $< 1 \text{ ms}$ 。

4.2 听牌检测

手牌数为 $3n + 1$ 时, 枚举所有可能的候补牌(优化: 仅搜索手牌已有花色 ± 2 范围, 约 12–18 种), 对每种候选模拟加入后调用胡牌判定。若任意候选能胡则听牌成立。

4.3 NPC 出牌策略

AI 出牌采用启发式权重评估:

- 听牌优先: 遍历手牌, 若丢弃某张后可直接听牌, 则优先打出
- 价值排序: 孤张 > 边张 > 散搭 > 对子 > 将对 > 王牌(绝不丢弃)
- 安全考量: 参考弃牌池避免打出炮牌
- 吃碰决策: 评估是否改善手牌结构

4.4 手牌拖拽与同步

单机模式

手牌顺序由拖拽操作直接修改, 发牌和摸牌时新牌追加至数组末尾(不排序), 所有游戏操作使

用 `splice` 精确保留顺序。

联机模式

客户端的拖拽顺序通过计数差集算法与服务端状态合并: 比较客户端和服务端牌的频数分布, 仅移除多余的牌(按客户端位置删除), 新增的牌(追加至末尾), 其余牌保持拖拽后的顺序不变。

5 多人在线基础设施

5.1 容错机制

断线重连: WebSocket 断开后采用指数退避策略重连: $1\text{s} \rightarrow 2\text{s} \rightarrow 4\text{s} \rightarrow \dots \rightarrow 128\text{s}$, 最多 8 次。房间号通过 URL 参数 `?auto_join=` 保存, 重连后自

死锁预防: 服务端分发操作提示后启动 10 秒超时计时器。若超时前仍有玩家未响应, 强制清除挂起状态并推进回合, 确保游戏不会因网络问题永久卡死。

心跳机制: 客户端每 10 秒发送 ping, 服务端立即回复 pong。往返时间 Δ 映射为游戏界面左上角的 4 格信号指示器。

服务端权威: 所有联机操作均由服务端校验。客户端仅做本地预览, 服务端的 `server/game/` 目录包含与前端 `core/` 完全相同的游戏引擎副本, 确保判定一致性。

5.2 网络延迟实测

6 跨平台兼容性

6.1 iOS 适配详情

iOS Safari 施加多项限制: (1) `getUserMedia` 需 HTTPS (GitHub Pages 已满足); (2) `AudioContext` 启动时处于 `suspended` 状态, 需在用户手势回调中显式 `resume()`; (3) `click` 事件仅在有隐式交互角色的元素上触发 (`<button>`、`<a href>`); (4) 软件键盘会破坏 `position:fixed` 的基准参考系。

TJMJ 中所有交互元素均实现为 `<button>` 并重置默认样式。`AudioContext` 采用单例模式, 首次用户交互时创建并恢复。触屏处理手动管理

`preventDefault()` 来抑制 iOS 的 350ms 双击缩放延迟。

6.2 Android 适配详情

Android Chrome 的动态地址栏导致 100vh 在地址栏显示/隐藏时发生跳动。TJMJ 使用 100dvh (动态视口高度) 替代, 并为旧浏览器保留 100vh 降级。此外, Android 碎片化的硬件生态导致 WebRTC 表现参差不齐, Twilio TURN 中继为直接 P2P 连接失败的场景提供了可靠后备。

7 性能评测

语音质量主观评估: TURN 中继条件下 4 人通话 MOS 平均分 3.8/5.0。P2P 直连音频延迟约 40ms, TURN 中继约 120ms。

8 讨论与展望

8.1 当前局限

1. **P2P 网状拓扑**: 4 人时需管理 $\mathcal{O}(N^2) = 6$ 条连接, 扩展性受限。引入 SFU 可降至 $\mathcal{O}(N)$
2. **单文件架构**: `App.vue` 2600+ 行, 维护成本随规模增长
3. **启发式 AI**: NPC 策略缺乏深度学习带来的战略高度
4. **音效缺失**: 牌名播报依赖 TTS 降级, 34 个预录制 mp3 待补充

8.2 未来方向

- **DRL 麻将智能体**: 基于 PPO + 自我对弈 + MCTS 训练超人类水平的 AI 玩家
- **分布式服务器**: 引入 Redis Pub/Sub 实现跨实例状态同步, 支持数千并发房间
- **PWA 离线体验**: Service Worker 缓存 + Web App Manifest, 实现可安装的离线应用
- **回放与观战**: 操作日志存储 + 时序回放, 支持多视角自动切换

参考资料

1. RFC 6455: The WebSocket Protocol, IETF, 2011.
2. WebRTC 1.0: Real-Time Communication Between Browsers, W3C, 2021.
3. Vue.js 3.5 官方文档, <https://vuejs.org/>, 2025.
4. Vite 8.0 官方文档, <https://vitejs.dev/>, 2025.
5. Node.js v20 文档, OpenJS Foundation, 2025.
6. OpenMajiang 项目, <https://gitee.com/mirrors/OpenMajiang>.
7. Twilio Network Traversal Service, <https://www.twilio.com/stun-turn>, 2025.
8. V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, 2015.
9. D. Silver et al., “Mastering the game of Go with deep neural networks and tree search,” Nature, 2016.